

This handout includes space for every question that requires a written response. Please feel free to use it to handwrite your solutions (legibly, please). If you choose to typeset your solutions, the —README.md— for this assignment includes instructions to regenerate this handout with your typeset $\text{\LaTeX}$ solutions.

1.b Behavior Cloning Results

Both environments were tested with identical hyperparameters for fair comparison:

- Network: 2 hidden layers, 64 units each
- Training steps: 1000 gradient updates
- Learning rate: $5e-3$
- Episode length: 1000
- Eval batch size: 10000 (for multiple rollouts)

Environment	Expert Return	BC Mean	BC Std	% of Expert
Ant-v4 (working)	4713.65	1551.48	993.03	32.9%
Walker2d-v4 (not working)	5566.85	295.01	341.21	5.3%

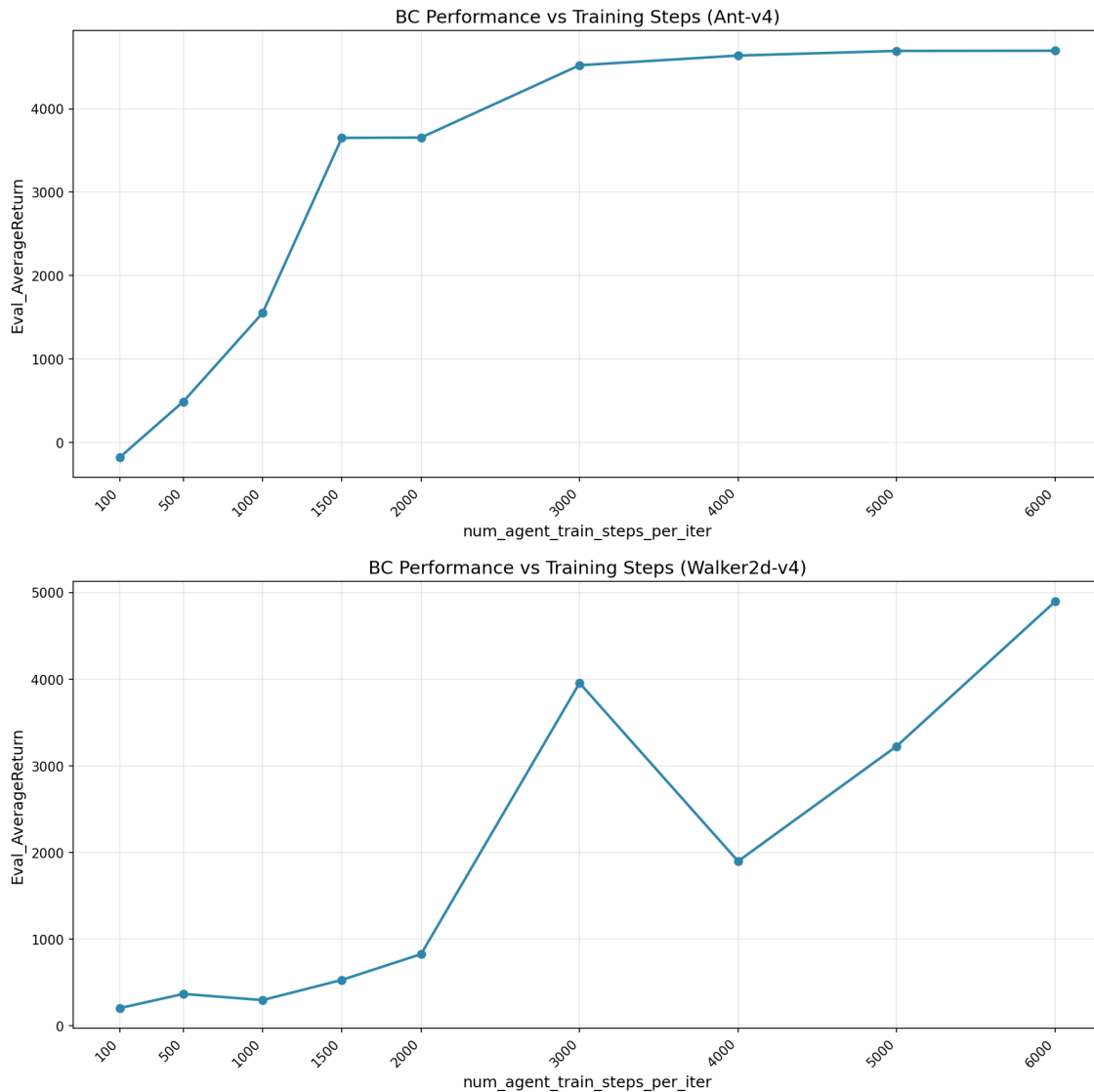
Analysis: BC achieves $>30\%$ on Ant-v4 but fails on Walker2d-v4 (only 5.3%). The high standard deviation in Walker2d (341.21) relative to the BC mean return indicates inconsistent behavior, with episodes often terminating early (avg episode length: 126 vs 1000 for expert). As we show in the experiment section below the Walker2d task takes for more gradient updates in order to learn the expert policy.

1.c

Hyperparameter: `num_agent_train_steps_per_iter`

Rationale: This hyperparameter controls how many gradient updates the policy network receives during training. It directly affects the underfitting/overfitting trade-off: too few steps means the policy hasn't learned the expert behavior, while too many steps could lead to overfitting on the limited expert data.

Experiment: Tested on Ant-v4 and Walker2d-v4 with values: 100, 500, 1000, 1500, 2000, 3000, 4000, 5000, 6000.



Results: For Ant-v4 performance increases rapidly from 100 to 3000 steps, then plateaus near expert level (~4700). At 100 steps, the policy is severely underfit (negative return). No overfitting was observed up to 6000 steps, suggesting the expert dataset is sufficient for this task.

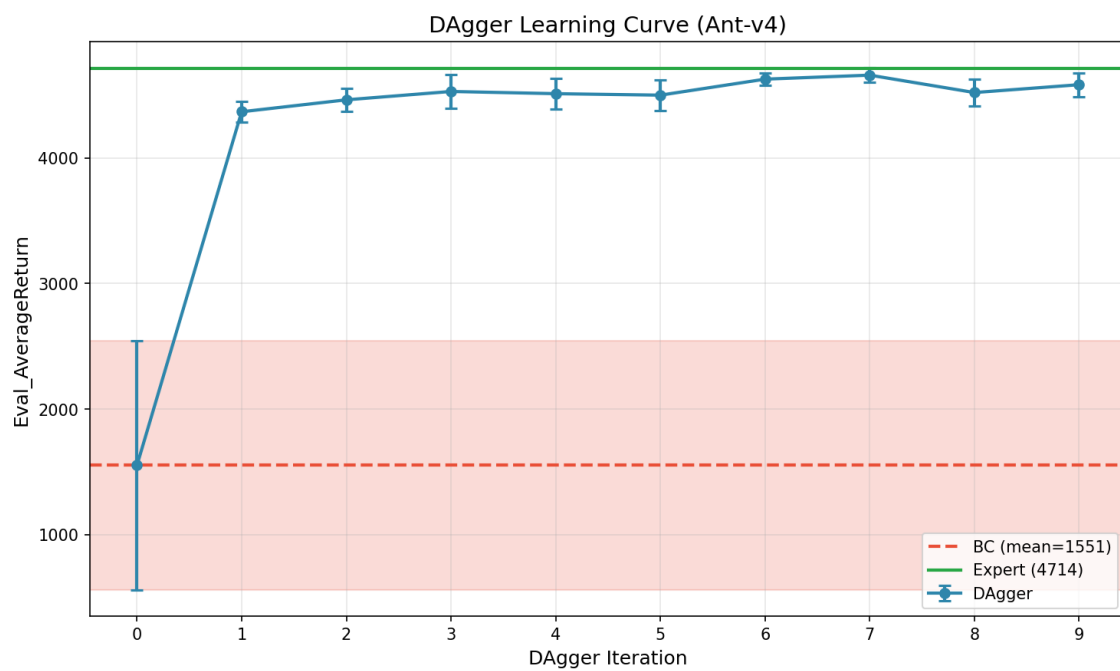
For Walker2d-v4 it takes for more training steps per iteration to improve the evaluation performance to get near expert performance.

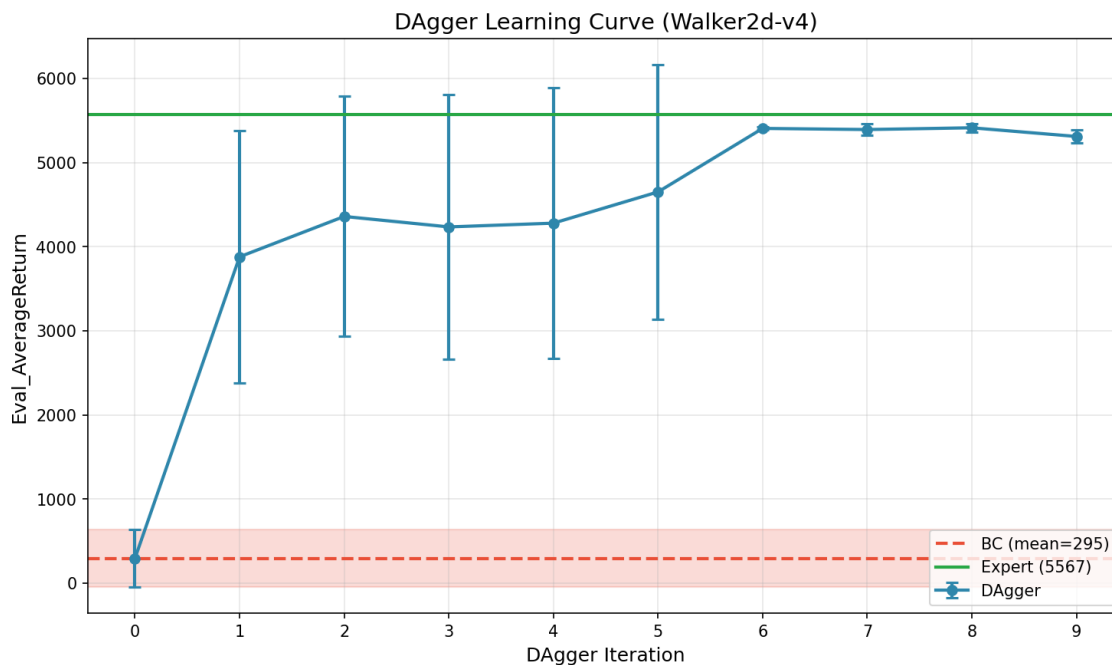
2.b

DAGger Results

DAGger was run on both Ant-v4 and Walker2d-v4 with identical hyperparameters:

- Network: 2 hidden layers, 64 units each
- Training steps per iteration: 1000
- Learning rate: 5e-3
- Batch size: 1000 (new data per iteration)
- Eval batch size: 5000
- Number of DAGger iterations: 10

Ant-v4 Learning Curve:**Walker2d-v4 Learning Curve:**



Environment	Expert	BC (iter 0)	DAGger Final	% of Expert
Ant-v4	4713.65	1551.48 ± 993.03	4583.09 ± 93.76	97.2%
Walker2d-v4	5566.85	295.01 ± 341.21	5310.39 ± 76.14	95.4%

Analysis: DAGger dramatically improves upon BC for both environments:

- **Ant-v4:** BC achieves only 33% of expert; DAGger reaches 97% after just 1 iteration, plateauing near expert level.
- **Walker2d-v4:** BC completely fails (5% of expert) with 1000 training steps per iteration, but DAGger recovers to 95% by iteration 6-7. The high variance in early iterations (1-5) shows the policy is still learning balance; variance drops dramatically once stable walking is achieved.

DAGger's interactive data collection addresses BC's distribution shift problem by querying the expert on states the learned policy actually visits, enabling recovery from mistakes.